

UNIVERSITATEA DIN BUCUREȘTI

FACULTATEA DE
MATEMATICĂ ȘI INFORMATICĂ

SPECIALIZAREA INFORMATICĂ

Lucrare de licență

GENERARE SECVENȚIALĂ DE
DESENE VECTORIALE PRINTR-O
BUCLĂ ARTIST–CRITIC

Absolvent

Mădălin Ioana

Coordonator științific

Conf. Dr. Ciprian Păduraru

București, iulie 2026

Rezumat

Lucrarea propune un sistem care generează desene vectoriale (SVG) pas cu pas, printr-o buclă iterativă cu doi agenți (Artist–Critic). La fiecare iterație, un model mare de limbaj (Artistul) scrie cod SVG, imaginea randată este evaluată de un model viziune–limbaj (Criticul), iar feedbackul localizat al acestuia ghidează pasul următor. Problema pe care o rezolvăm este regresia: într-o buclă naivă, Artistul strică frecvent elemente desenate corect la pașii anteriori. Soluția este un mecanism de *region lock* impus prin cod. Restrângând fiecare revizuire la o singură regiune numită explicit de Critic, sistemul garantează că restul desenului rămâne neatins, deci desenul nu se poate decât îmbunătăți de la un pas la altul. Sistemul rulează fie pe un backend cloud (Google), fie pe unul local (Ollama), și arată întregul proces, pas cu pas, printr-o interfață web. Evaluarea arată că bucla face desenele mai ușor de recunoscut decât o singură generare directă, însă rezultatul depinde mult de cât de capabile sunt modelele folosite.

Abstract

This thesis proposes a system that generates vector drawings (SVG) step by step, through an iterative two-agent loop (Artist–Critic). At each iteration a large language model (the Artist) writes SVG code, the rendered image is evaluated by a vision–language model (the Critic), and the Critic’s localised feedback guides the next step. The problem we solve is regression: in a naive loop, the Artist frequently damages elements that were drawn correctly in earlier steps. The solution is a code-enforced *region lock* mechanism. By restricting each revision to a single region explicitly named by the Critic, the system guarantees that the rest of the drawing stays untouched, so the drawing can only improve from one step to the next. The system runs either on a cloud backend (Google) or a local one (Ollama), and shows the whole process, step by step, through a web interface. Evaluation shows that the loop makes drawings easier to recognise than a single direct generation, although the result depends heavily on how capable the underlying models are.

Cuprins

1	Introducere	5
1.1	Context și motivație	5
1.2	Problema abordată și ipoteza	5
1.3	Contribuții	6
1.4	Structura lucrării	6
2	Fundamente teoretice și lucrări înrudite	7
2.1	Modele mari de limbaj și arhitectura Transformer	7
2.2	Modele multimodale de tip viziune–limbaj	7
2.3	Grafica vectorială scalabilă (SVG)	7
2.4	Stadiul actual al cercetării	8
2.5	Poziționarea lucrării curente	8
3	Arhitectura sistemului și implementare	9
3.1	Privire de ansamblu a buclei Artist–Critic	9
3.2	Artistul: generarea codului SVG	10
3.3	Randarea și animația pas cu pas	10
3.4	Criticul: evaluare vizuală și feedback localizat	11
3.5	Region lock: garantarea îmbunătățirii monotone	11
3.6	Orchestrarea și fluxul de control	12
3.7	Implementare: backend-uri, modele și interfață	12
4	Evaluare și rezultate	13
4.1	Metodologia evaluării	13
4.2	Cele trei regimuri ale buclei	13
4.3	De ce unele prompturi ies complete din prima trecere	16
4.4	O singură trecere versus bucla completă	17
4.5	Comparația între modele	18
4.6	Efectul mecanismului de region lock	19
4.7	Sinteza rezultatelor și limitări	20
5	Concluzii și direcții viitoare	22
5.1	Concluzii	22
5.2	Limitări	22
5.3	Direcții viitoare	22

1. Introducere

1.1 Context și motivație

Ultimii ani au adus progrese rapide în generarea automată de imagini, de la rețelele generative adversariale (GAN) la modelele de difuzie precum Stable Diffusion [8] sau DALL·E. Aproape toate produc însă imagini *raster* (matrice de pixeli), dintr-o singură trecere. Generarea de *grafică vectorială* (SVG) e o problemă diferită și mai grea: într-o imagine raster o greșeală rămâne strict locală, pe când un desen vectorial cere înțelegere structurală și geometrică, fiindcă modelul trebuie să decidă unde „pune creionul”, cum curbează o linie și când închide o formă.

La asta se adaugă o motivație de *proces*: un om nu desenează dintr-odată, ci construiește desenul treptat și îl corectează pe parcurs. Un sistem care desenează *secvențial* și explică fiecare etapă are valoare didactică, fiindcă arată *cum* se construiește un desen, nu doar rezultatul final.

Modelele mari de limbaj (LLM) pot scrie direct cod SVG, deci pot juca rolul de „artist”. Două lucruri fac însă sarcina grea. Întâi, raționamentul spațial pornind doar de la text e slab, iar modelele greșesc des coordonatele [10]. Apoi, un LLM textual e „orb” la propriul rezultat: odată ce a emis codul, nu vede imaginea randată și nu o poate corecta. Ideea de bază a lucrării este *închiderea buclei*: un al doilea model, de viziune, se uită la desenul randat și oferă feedback, transformând generarea într-un dialog *Artist–Critic*.

Tocmai aici apare problema tehnică principală. Într-o buclă de feedback naivă, desenul nu se îmbunătățește monoton de la o iterație la alta: aplicând sugestiile Criticului, Artistul strică adesea părți care erau deja corecte (regresie). Obiectivul central al lucrării este să garanteze că fiecare iterație *adaugă* ceva, fără să distrugă progresul de până atunci.

1.2 Problema abordată și ipoteza

Lucrarea pornește de la o întrebare: *poate un model de limbaj pur textual să-și îmbunătățească iterativ un desen vectorial, ghidat de un model de viziune separat, astfel încât desenul să progreseze monoton de la o iterație la alta?*

Ipoteza lucrării este că o buclă închisă Artist–Critic produce desene care se îmbunătățesc de la un pas la altul și rămân interpretabile ca o construcție pas cu pas, dacă sunt îndeplinite două condiții: Criticul dă feedback *localizat* (o singură corecție, legată de o regiune precisă a desenului), iar un mecanism de *blocare a regiunilor* (*region lock*) impus prin cod împiedică modificarea zonelor care nu au fost vizate.

Ca să verificăm ipoteza, proiectăm bucla Artist–Critic peste SVG, definim un Critic cu feedback localizat (verdict, scor, acțiune și regiune-țintă), implementăm mecanismul

de region lock care forțează îmbunătățirea monotonă și evaluăm sistemul pe un set fix de prompturi, atât pe cazuri de succes, cât și pe cazuri de eșec.

1.3 Contribuții

Principalele contribuții ale lucrării sunt:

- Bucla Artist–Critic pentru desen vectorial secvențial, în care un LLM textual generează cod SVG, iar un model de viziune îl evaluează și îl ghidează iterativ prin feedback localizat.
- Mecanismul de region lock impus prin cod, contribuția metodologică centrală: restrângând fiecare revizuire la o singură regiune numită de Critic, sistemul transformă o buclă instabilă într-una care progresează monoton.
- O implementare open-source, care rulează complet local modele open-weights pe hardware de consum și arată procesul pas cu pas prin animație și explicații în limbaj natural.

1.4 Structura lucrării

Capitolul 2 prezintă fundamentele (LLM-uri, modele multimodale, SVG) și lucrările înrudite. Capitolul 3 descrie arhitectura și implementarea, cu accent pe region lock. Capitolul 4 prezintă evaluarea experimentală, iar Capitolul 5 concluziile și direcțiile viitoare.

2. Fundamente teoretice și lucrări înrudite

Acest capitol introduce conceptele de care avem nevoie (modelele de limbaj, modelele multimodale și formatul SVG) și trece în revistă principalele direcții din generarea de desene, ca să situăm lucrarea față de ele.

2.1 Modele mari de limbaj și arhitectura Transformer

Un *model mare de limbaj* (*Large Language Model*, LLM) este antrenat pe cantități uriașe de text [1] ca să continue o secvență, token cu token. Arhitectura dominantă e *Transformer* [9], al cărei mecanism de *atenție* poate lega direct cuvinte aflate oriunde în secvență, fără recurență.

Pentru lucrare contează un singur lucru: un LLM tratează orice secvență de simboluri ca text, fie că e limbaj natural, fie cod. Iar codul SVG este o reprezentare XML, textuală, deci poate fi generat token cu token, ca o propoziție. Așa poate un LLM să producă desene scriindu-le direct codul, adică să joace rolul de *Artist* din sistemul propus.

2.2 Modele multimodale de tip viziune–limbaj

Un clasificator de imagini obișnuit alege doar dintre categoriile văzute la antrenament. *Modelele multimodale* pun în schimb textul și imaginea în aceeași reprezentare, ca să poată fi comparate direct; CLIP [7] a popularizat ideea, antrenând un encoder de imagini și unul de text să apropie perechile imagine–descriere potrivite și să le depărteze pe cele nepotrivite.

Pe această bază au apărut *modelele viziune–limbaj* (*Vision–Language Models*, VLM) antrenate să urmeze instrucțiuni [5]: primesc deodată o imagine și un text și răspund în limbaj natural. Un astfel de model poate deci să *descrie* ce vede și să evalueze imaginea față de o cerință, adică exact ce ne trebuie pentru rolul de *Critic*.

2.3 Grafica vectorială scalabilă (SVG)

Spre deosebire de o imagine raster, care reține o valoare per pixel, un fișier SVG (*Scalable Vector Graphics*) ține *rețeta* desenului: linii, curbe Bézier, cercuri sau poligoane cu coordonatele lor, scrise ca XML. E potrivit pentru proiect din două motive: fiind text, un model de limbaj îl poate scrie ca pe o propoziție; și, fiecare formă fiind un obiect separat, desenul e *independent de rezoluție* și editabil element cu element: o formă deja trasată poate rămâne neatinsă sau poate fi modificată separat. Pe asta se bazează sistemul când blochează regiunile corecte ale unui desen (Capitolul 3).

Reversul este că generarea corectă de SVG cere raționament *geometric*. Într-o imagine raster, o eroare e strict locală și are un impact vizual limitat; într-o cale SVG, o singură

coordonată greșită poate deplasa sau deforma o formă întreagă [2].

2.4 Stadiul actual al cercetării

Generarea de desene a trecut prin mai multe paradigme, fiecare cu limitări care motivează abordarea de față.

Reperul în generarea secvențială de desene a fost stabilit de Ha și Eck prin *Sketch-RNN* [3], care vede desenul ca o serie de acțiuni ale creionului (Δx , Δy , stare), modelate cu rețele recurente sequence-to-sequence. Modelul pierde însă firul pe secvențe lungi și nu înțelege semantica desenului. DeepSVG [2] obține desene mai coerente la nivel global printr-o rețea transformer ierarhică, ce reprezintă întregul desen ca un singur vector de caracteristici, din care poate apoi genera pictograme noi sau treceri graduale între două pictograme existente. Rămâne însă o generare într-o singură decodare: desenul nu se construiește pas cu pas și nu se corectează pe baza imaginii randate.

Odată cu succesul modelelor de difuzie, cercetătorii le-au adaptat pentru grafică vectorială. VectorFusion [4] pornește de la curbe Bézier aleatoare, le randează raster și ajustează iterativ parametrii prin *Score Distillation Sampling*. SVGDreamer [11] extinde această abordare cu o vectorizare ghidată semantic și un score distillation bazat pe particule, obținând SVG-uri mai curate și mai editabile. Limitele rămân aceleași pentru problema noastră: optimizare lentă, nesecvențială, pe imaginea globală finală, nu pe codul SVG.

Cea mai recentă direcție pune modelele de limbaj să scrie direct cod SVG. *LLM4SVG* [10] arată că modelele generale se descurcă greu cu coordonatele spațiale și introduce tokeni semantici speciali care ajută modelul să „vizualizeze” geometria. Este abordarea cea mai apropiată de a noastră și confirmă că LLM-urile pot fi „artiști” dacă sunt ghidate cum trebuie. Rămâne totuși o generare într-o singură trecere, fără autocorecție pe baza rezultatului vizual.

2.5 Poziționarea lucrării curente

Sistemul propus se diferențiază de aceste direcții prin *bucla multimodală cu feedback* (Artist–Critic) [6]. Sketch-RNN și DeepSVG construiesc o reprezentare a desenului, dar sunt „oarbe” la rezultatul randat și nu se autocorectează. VectorFusion și SVGDreamer optimizează o imagine finală prin difuzie, fără construcție secvențială și fără explicații. Iar LLM4SVG generează cod dintr-o singură trecere, fără corecție vizuală. Sistemul de față cuplează în schimb un LLM textual (codul SVG) cu un model de viziune (corecția pe baza desenului randat), impune prin cod îmbunătățirea monotonă (*region lock*) și explică fiecare pas în limbaj natural.

3. Arhitectura sistemului și implementare

Acest capitol descrie arhitectura sistemului propus și deciziile de implementare. Începem cu o privire de ansamblu a buclei Artist–Critic, apoi luăm pe rând fiecare componentă, cu accent pe mecanismul de *region lock*, în jurul căruia se construiește lucrarea.

3.1 Privire de ansamblu a buclei Artist–Critic

Sistemul este o buclă cu doi agenți și patru etape, rulată până la un buget maxim de iterații (Figura 3.1). Se poate opri mai devreme doar când Criticul acceptă clar: verdict **accept**, scor cel puțin 9 / 10 și toate părțile principale marcate ca prezente. Pornind de la un prompt în limbaj natural:

1. Artistul, un model de limbaj textual, emite un document SVG, împreună cu un raționament și etichete pentru fiecare trăsătură desenată;
2. randarea transformă SVG-ul într-o imagine PNG (biblioteca `cairosvg`);
3. Criticul, un model viziune–limbaj, se uită la imagine și produce un obiect JSON structurat: un scor, un verdict, starea fiecărei părți și, esențial, *o singură* corecție localizată (o acțiune și o regiune-țintă);
4. corecția se întoarce la Artist pentru iterația următoare, sub controlul mecanismului de region lock.

După acceptare sau după ce se epuizează bugetul de iterații, sistemul returnează desenul SVG final, împreună cu istoricul complet și o animație a desenului în construcție.

De reținut că Artistul nu vede niciodată imaginea randată: el primește numai promptul, desenul anterior și feedbackul în limbaj natural al Criticului.

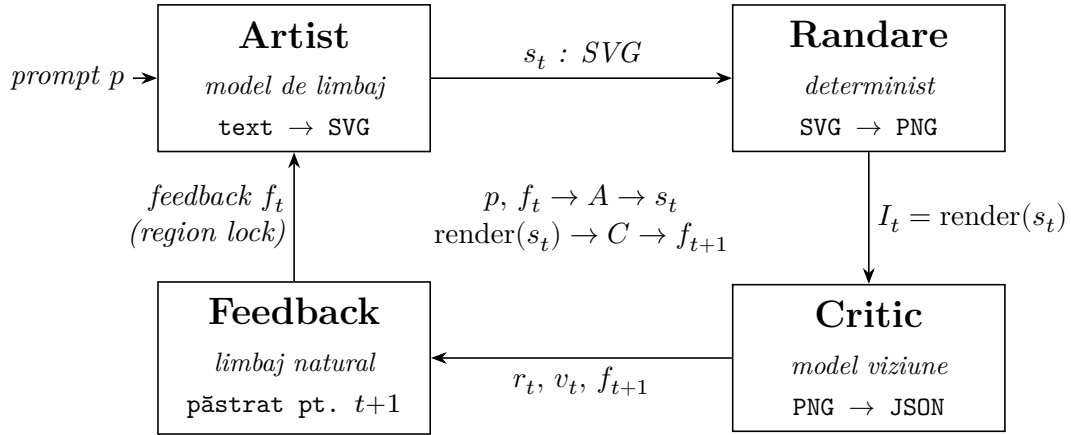


Figura 3.1: Schema buclei Artist–Critic. Promptul intră în Artist; fiecare iterație produce un SVG (s_t), randat în imagine (I_t) și evaluat de Critic, care întoarce scor, verdict și feedback localizat (f_{t+1}). Feedbackul revine la Artist sub region lock; la acceptare, ciclul se oprește și se returnează SVG-ul final.

3.2 Artistul: generarea codului SVG

Artistul primește promptul (la prima iterație) sau promptul împreună cu desenul anterior și corecția Criticului (la revizuiți) și returnează un JSON cu codul SVG, un raționament și o listă de *etichete de pas*, câte una per trăsătură. Fiecare cale capătă un identificator **step-N**, unde N dă ordinea desenării. Cu numerotarea asta, fiecare stroke poate fi identificat separat, lucru de care avem nevoie atât pentru animația pas cu pas, cât și pentru region lock (Secțiunea 3.5).

Desenele folosesc un canvas fix de 512×512 și un stil de schiță cu linie neagră, peste care codul aplică o ușoară perturbație (*wobble*) ca liniile să pară trase de mână. Fiindcă modelele locale scot uneori SVG invalid (attribute fără ghilimele, taguri neînchise), toată ieșirea trece printr-un pas de validare și reparare: în loc să eșueze, un parser de recuperare salvează ce se poate.

3.3 Randarea și animația pas cu pas

Randarea are trei roluri. Produce imaginea PNG afișată în interfață; generează *cadrele progresive* (afișând pe rând căile **step-1**, **step-2**, ..., se obține o animație a desenului în construcție); și produce imaginea trimisă Criticului, randată la rezoluție mai mare, pe fundal alb și cu linia îngroșată, fiindcă modelele de viziune interpretează greu schițele cu linii subțiri.

De la a doua iterație, Criticul nu mai primește o singură imagine, ci o *foaie de comparație* în trei panouri: desenul anterior (stânga), desenul curent (mijloc) și stroke-urile nou adăugate (dreapta). Așa poate verifica dacă trăsătura cerută data trecută chiar a apărut, în loc să rejudece tot desenul de la zero.

3.4 Criticul: evaluare vizuală și feedback localizat

Criticul este un model viziune–limbaj căruia i se cere să returneze doar un obiect JSON cu schema din Listingul 3.1. Câmpurile importante sunt scorul de recognoscibilitate (1–10), starea părților (`part_status`, fiecare parte etichetată *missing*, *present*, *weak* sau *malformed*) și, mai ales, perechea `action` / `target_region`, care spune *ce* are voie Artistul să modifice și *unde*.

```
{
  "verdict":      "accept" | "revise",
  "score":        <intreg 1-10>,
  "part_status":  ["coada: malformed", "urechi: missing", ...],
  "feedback_for_artist": "<o judecata vizuala + o singura actiune>",
  "action":       "add" | "redraw_element" | "redraw_all",
  "target_region": "<o singura regiune, ex. 'window', 'whiskers'>"
}
```

Listing 3.1: Schema JSON returnată de Critic (câmpurile principale).

Promptul de sistem impune un *contract de o singură schimbare*. La fiecare iterație, Criticul judecă întâi trăsătura cerută data trecută, marchează drept bune părțile care se văd deja clar și cere exact o trăsătură nouă, în ordinea firească a unui desen: întâi conturul, apoi părțile mari și definitorii, la final detaliile mici. Nu are voie să ceară refacerea întregului desen doar ca să-l mai șlefuiască, iar o parte care arată deja bine rămâne neatinsă. Contractul previne oscilația și ține progresul într-o singură direcție.

3.5 Region lock: garantarea îmbunătățirii monotone

Problema de bază a oricărei bucle de feedback naive este *regresia*: când Artistul rescrie tot SVG-ul la fiecare pas, strică frecvent părți care erau deja corecte. Nu ne bazăm pe speranța că modelul va păstra ce e bun, ci *impunem asta prin cod*.

Mecanismul se folosește de numerotarea `step-N` și de perechea `action/target_region`. După ce Artistul produce un SVG nou, codul îl compară cale cu cale cu desenul anterior și aplică regula din Listingul 3.2, în funcție de acțiunea cerută de Critic:

- **add**, cazul implicit: *toate* căile existente sunt blocate, iar desenul anterior este păstrat neschimbat; din ieșirea Artistului se păstrează doar stroke-urile noi (cu identificatori `step` ulteriori). Așa, o iterație nu poate decât să *adauge*.
- **redraw_element**: pot fi modificate doar calea (sau căile) regiunii-țintă; orice altă cale a cărei geometrie s-a schimbat este readusă la valoarea anterioară.
- **redraw_all**, singura excepție: mecanismul este dezactivat. Îl rezervăm cazului rar în care desenul nu conține încă nicio formă coerentă.

```
pentru fiecare cale step-N din desenul_anterior:
    daca step-N este in regiunea-tinta:
        pastreaza versiunea noua          # modificare permisa
    altfel daca Artistul i-a schimbat geometria:
        revino la geometria anterioara    # drift nepermis -> anulat
caile noi (cu id absent anterior) trec neschimbate
```

Listing 3.2: *Regula de blocare a regiunilor (pentru `redraw_element`; pentru `add`, regiunea-țintă este vidă, deci toate căile existente sunt blocate).*

Prin construcție, orice parte pe care Criticul nu o numește explicit rămâne *neschimbată* față de iterația anterioară: codul ei SVG este identic, caracter cu caracter. La fiecare iterație, sistemul notează câte căi a păstrat și câte a readus la forma anterioară (*preservation report*), pe care le agregă apoi într-o rată de păstrare folosită în evaluare (Capitolul 4).

Garanția are însă o limită pe care o spunem deschis: region lock-ul asigură că geometria *blocată* nu se degradează, dar nu garantează că trăsătura *nou adăugată* iese bine. Cât de reușită e fiecare adăugare rămâne o întrebare empirică, pe care o tratăm în capitolul de evaluare.

3.6 Orchestrarea și fluxul de control

Bucula rulează cu un buget maxim de iterații, dar se oprește singură când Criticul acceptă clar. Ca să nu se oprească prematur, verdictul *accept* nu e de ajuns: orchestratorul verifică determinist că scorul e cel puțin 9 / 10, că `feedback_for_artist` nu mai cere o revizie, că `target_region` nu mai indică nicio regiune de corectat și că toate intrările din `part_status` sunt *present*. Dacă lipsește vreuna, acceptarea e tratată ca ambiguă, iar bucla continuă cu o revizie localizată.

Pe lângă region lock-ul geometric, orchestratorul ține și o protecție la nivelul planului de desen, nu al geometriei: o parte raportată ca prezentă rămâne „blocată” pentru restul rulării, ca modelul de viziune, instabil de la un pas la altul, să nu mai ceară o trăsătură deja desenată. La final, calculează metricile de evaluare: traiectoria scorurilor, deltele, rata de păstrare și timpii pe etapă.

3.7 Implementare: backend-uri, modele și interfață

Sistemul poate folosi un backend diferit *pentru fiecare rol*: rulare locală prin Ollama sau în cloud prin Google, alese din variabile de mediu. Artistul are nevoie de un model care generează SVG coerent, iar Criticul de un model multimodal care evaluează imaginea. Rulările principale folosesc `gemma4:31b` pentru Artist și `internv13.5:14b` pentru Critic; comparația cu `gemma4:26b`, `qwen2.5:14b` și `qwen2.5-coder:14b` este în Capitolul 4.

Interfața are mai ales un rol ilustrativ. Un serviciu FastAPI trimite evenimentele buclei (generare, randare, critică) către o pagină web pe măsură ce apar, iar pagina redă în timp real animația pas cu pas, raționamentul Artistului și feedbackul Criticului. Așa, utilizatorul vede tot procesul de construcție și autocorecție, nu doar rezultatul final.

4. Evaluare și rezultate

Acest capitol evaluează comportamentul sistemului Artist–Critic printr-o serie de rulări controlate pe prompturi simple de desen. Ne interesează patru întrebări practice: *(i)* în ce condiții bucla de feedback chiar îmbunătățește desenul; *(ii)* dacă bucla completă aduce un câștig față de o singură trecere; *(iii)* cât ajută mecanismul de region lock la păstrarea părților deja corecte; și *(iv)* cât de mult depinde rezultatul de modelul folosit. Raportăm atât cazurile în care bucla ajută clar, cât și cele în care nu prea are ce face.

În toate experimentele principale (în afară de comparația dintre modele), Artistul folosește *Gemma 4 31B*, iar Criticul *InternVL3.5 14B*, ambele modele rulate local, cu un buget de șase pași per prompt. La fiecare pas, Criticul întoarce un scor (1–10), o acțiune și un feedback textual; bucla se oprește mai devreme doar la o acceptare clară, adică scor cel puțin 9 / 10 și toate părțile principale prezente (Capitolul 3).

4.1 Metodologia evaluării

Evaluarea se sprijină pe prompturi scurte, vizuale, pentru care e ușor de verificat dacă obiectele și atributele cerute apar în desen: „*a smiling sun*”, „*a cat with a curled tail*”, „*a coffee mug with a handle*”, „*a pair of headphones*”, „*a rocket with two fins, a round window, and flames*”, „*a house with a roof, a door, two windows, and a chimney*”, „*a sailboat on waves*” și „*a bicycle*”.

Scorurile nu sunt o măsură absolută a calității estetice, ci un semnal intern al modelului despre cât de bine se potrivește desenul cu promptul; problema acestui rol dublu al Criticului o discutăm în Secțiunea 4.7.

Analiza rulărilor scoate la iveală trei regimuri în care se comportă bucla, în jurul cărora e organizat capitolul: completare progresivă, când desenul pornește de la forma de bază și bucla adaugă pe rând părțile lipsă (pisica); rafinare, când obiectul e deja recognoscibil și bucla doar adaugă detalii secundare (barca); și oprire timpurie, când primul desen satisface deja promptul, iar Criticul acceptă fără alte iterații (racheta).

4.2 Cele trei regimuri ale buclei

4.2.1 Completare progresivă: „*a cat with a curled tail*”

Pisica arată cel mai clar utilitatea buclei. Pornind de la două ovale (cap și corp), Criticul cere pe rând urechi, ochi, mustăți, nas, gură, apoi lăbuțe, iar scorul crește de la 4 / 10 la 9 / 10 în șase pași, fără să se piardă forma de ansamblu, fiindcă fiecare pas adaugă o trăsătură peste cele deja blocate.

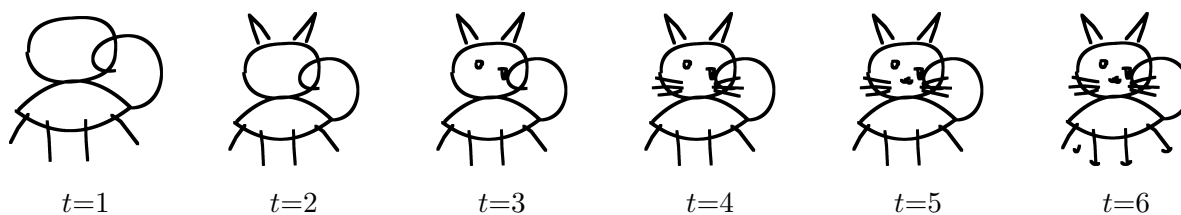


Figura 4.1: *Completare progresivă: evoluția desenului pentru „a cat with a curled tail”. Fiecare pas adaugă o trăsătură nouă peste cele anterioare.*

t	Scor	Acțiune	Critic note
1	4 / 10	add · ears	<i>You have the basic shapes down. Now let's add some character to the head.</i>
2	6 / 10	add · eyes	<i>The basic shape of the cat is all there. Now let's start adding some personality to the face.</i>
3	6 / 10	add · whiskers	<i>The basic shape is coming together well. Now let's add some whiskers to give it more character.</i>
4	7 / 10	add · nose and mouth	<i>The whiskers look great and really bring the face to life. Now let's add a little nose and mouth to finish the expression.</i>
5	8 / 10	add · paws	<i>The face looks great now. Let's give the cat some paws to finish the legs.</i>
6	9 / 10	accept · complete	<i>Great job completing the drawing!</i>

Fiecare instrucțiune (add ears, add eyes...) produce exact trăsătura cerută la pasul următor, fără redesenarea întregii imagini, adică tocmai comportamentul pentru care a fost proiectată bucla.

4.2.2 Rafinare: „a sailboat on waves”

Barca e recunoscutibilă din prima trecere (corp, catarg, vele; 7 / 10). Aici bucla nu mai completează obiectul, ci îl rafinează cu elemente secundare (un steag, hublouri, cârma), iar scorul urcă monoton de la 7 / 10 la 10 / 10, cu diferențe mai fine decât la pisică.

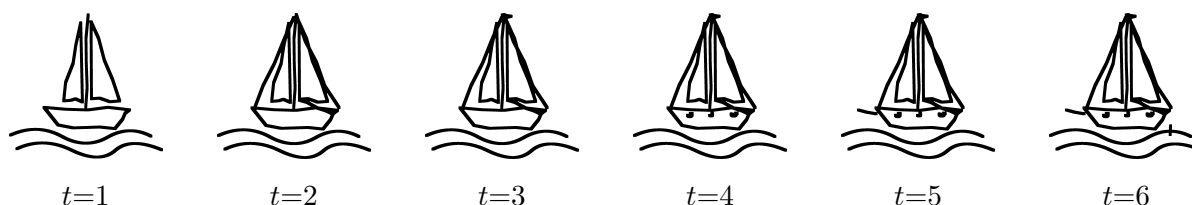


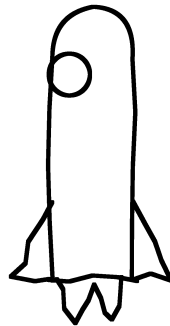
Figura 4.2: *Rafinare: evoluția desenului pentru „a sailboat on waves”. Obiectul este recunoscutibil de la $t = 1$, iar bucla adaugă detalii secundare.*

t	Scor	Acțiune	Critic note
1	7 / 10	add · rigging	<i>You have the main shapes down perfectly. Let's add some rigging to connect the sails and mast to the hull.</i>
2	7 / 10	add · flag	<i>You have the main structure down perfectly. Let's add a small flag at the top of the mast to give it some character.</i>
3	7 / 10	add · portholes	<i>You have all the main components in place. Let's add some small portholes to the side of the hull to give it more character.</i>
4	8 / 10	add · bowsprit	<i>The portholes look good and the boat is really taking shape. Let's add a bowsprit to the front of the hull.</i>
5	9 / 10	add · rudder	<i>The bowsprit looks good. The whole sketch is coming together nicely.</i>
6	10 / 10	accept · complete	<i>Great job completing the sketch!</i>

Regimul de rafinare arată că bucla nu strică un desen deja bun: trăsăturile noi se adaugă fără să afecteze structura, iar scorul nu scade pe parcurs.

4.2.3 Oprere timpurie: „a rocket with two fins, a round window, and flames”

La polul opus, racheta e acceptată din prima trecere: primul desen are deja corpul, fereastra, aripile și flăcările, deci Criticul dă 10 / 10 și bucla se oprește. Lipsa unei traiectorii iterative e tot un rezultat valid: orchestratorul nu face pași inutili când promptul e deja satisfăcut.



$t=1 \cdot 10 / 10 \cdot \text{acceptat}$

Figura 4.3: Oprere timpurie: „a rocket with two fins, a round window, and flames” este acceptată din prima trecere.

4.2.4 Când rafinarea aglomerează: „a bicycle”

Rafinarea nu e mereu un câștig net. Bicicleta e recognoscibilă de la $t = 1$ (două roți, cadru, ghidon), iar bucla adaugă componentele mecanice (pedale, lanț, spițe, cabluri de frână). Scorul crește de la 6 / 10 la 9 / 10, fiindcă Criticul punctează generos detaliile, dar trăsăturile se aglomerează (mai ales în față), iar desenul final e mai încărcat și mai greu de citit decât silueta inițială (Figura 4.4).

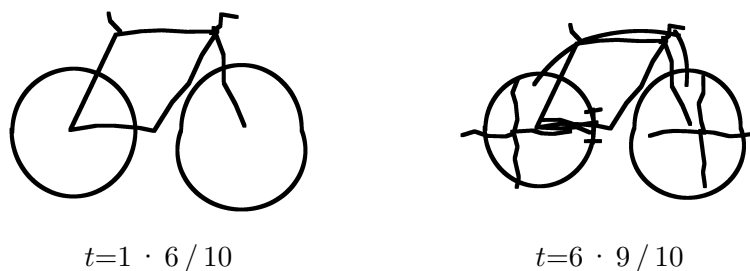


Figura 4.4: „a bicycle”: bucla adaugă pedale, lanț, spițe și cabluri de frână, iar scorul crește, dar detaliile aglomerează silueta inițială. Mai mult detaliu nu înseamnă neapărat un desen mai clar.

Buclo face exact ce i se cere, adică adaugă fără să distrugă structura, dar criteriul de oprire bazat pe scor nu prinde pierderea de lizibilitate, o limitare la care revenim în Secțiunea 4.7.

4.3 De ce unele prompturi ies complete din prima trecere

Racheta nu e o excepție izolată. Mai multe prompturi din setul principal dau, deja la $t = 1$, un desen aproape complet, iar bucla ajunge să *confirme* desenul, nu să-l construiască. Figura 4.5 pune alături prima și ultima iterație pentru trei astfel de cazuri (soarele, cana și căștile): deși scorul fluctuează, desenul final e aproape identic cu cel inițial, cu cel mult un detaliu în plus (aburul la cană, cablul la căști).

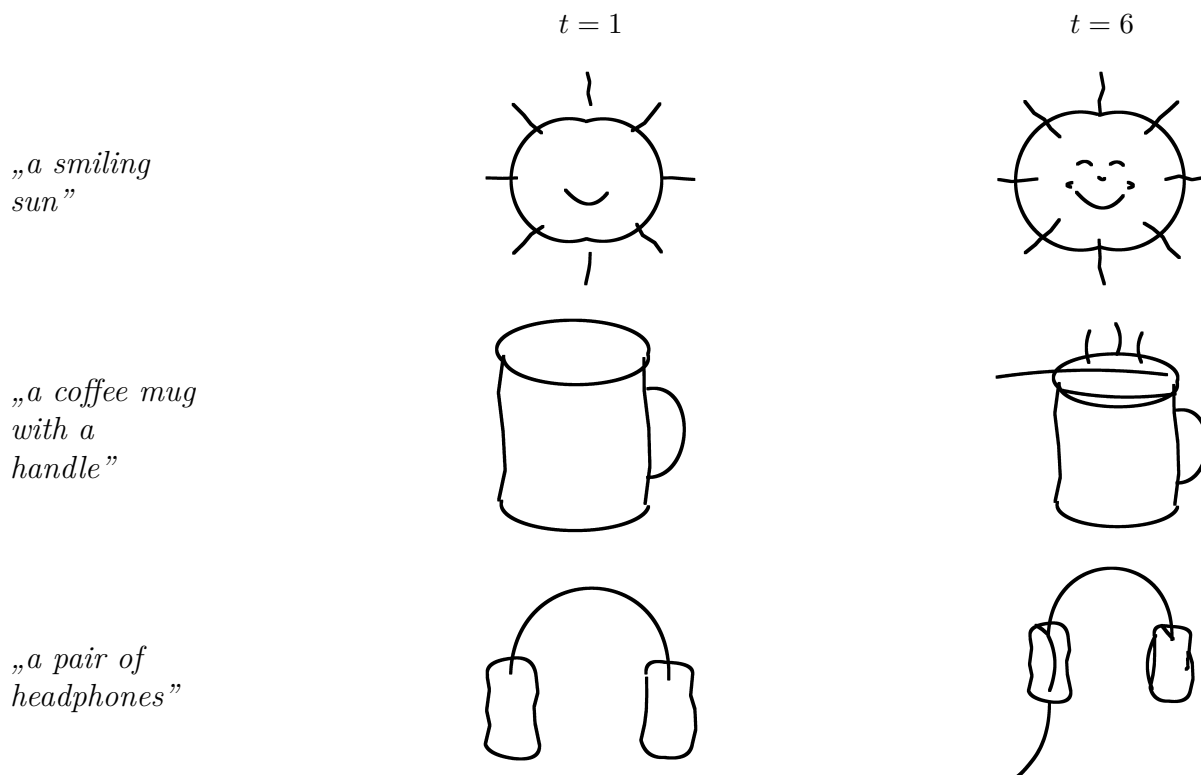


Figura 4.5: Prompturi care ies aproape complete din prima trecere: prima ($t = 1$) și ultima ($t = 6$) iterație sunt aproape identice. Buclo confirmă desenul și, cel mult, adaugă un detaliu secundar (aburul la cană, cablul la căști).

Comportamentul nu înseamnă că Artistul ignoră instrucțiunile: el chiar e instruit ca la prima iterație să deseneze doar forma de bază și să construiască de la formele mari spre detalii. Totul ține de tipul de prompt. La cele cu atribute listate („a house with a roof, a door, two windows, and a chimney”; „a rocket with two fins, a round window, and flames”), acoperișul, ușa sau ferestrele *sunt* chiar „părțile mari” pe care prima iterație oricum trebuie să le deseneze, iar bugetul de 3–6 căi ajunge pentru ele, deci un desen corect le conține de la $t = 1$. Această ordine contează abia când subiectul are părți *separabile* de detaliul ulterior, ca la pisică. Așadar convergența rapidă ține de prompt, nu e un eșec al Artistului: astfel de prompturi testează mai ales generarea într-o singură trecere, pe când robustețea *buclei* se vede pe subiecte care permit construcție incrementală.

4.4 O singură trecere versus bucla completă

Această ablație compară rulări independente, una cu o singură trecere și una completă cu feedback iterativ, pe subiecte care permit construcție incrementală, ca să vedem dacă prezența Criticului contează. Am păstrat exemplele cu diferență vizibilă.

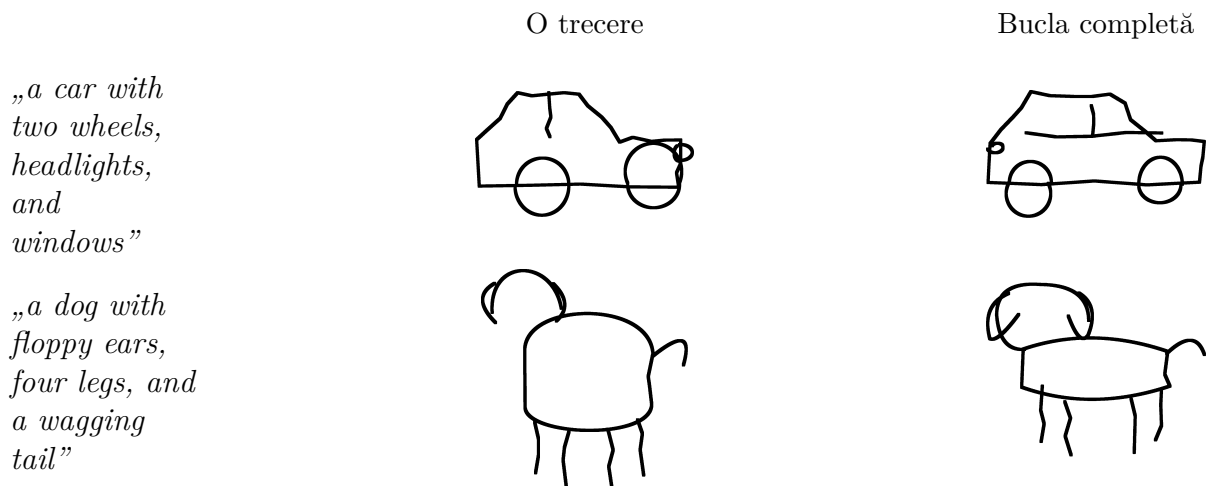


Figura 4.6: Comparație între o singură trecere și o buclă completă. La mașină, prima trecere are roți suprapuse și un contur neclar, pe care bucla le corectează; la câine, bucla clarifică raportul cap–corp și picioarele.

Prompt	O trecere	Bucla completă	Δ
„a car with two wheels, headlights, and windows”	7 / 10	9 / 10	+2
„a dog with floppy ears, four legs, and a wagging tail”	7 / 10	9 / 10	+2

Tabela 4.1: Scorurile pentru ablația o singură trecere versus bucla completă.

Feedbackul iterativ ajută mai ales când prima imagine e parțială sau dezechilibrată, ca la mașină, unde roțile suprapuse și conturul confuz din spate sunt reasezate de buclă.

4.5 Comparația între modele

Această ablație compară patru modele în rolul de Artist (*gemma4:31b*, *gemma4:26b*, *qwen2.5:14b* și *qwen2.5-coder:14b*), fiecare evaluat pe trei prompturi și trei iterații, cu același Critic.

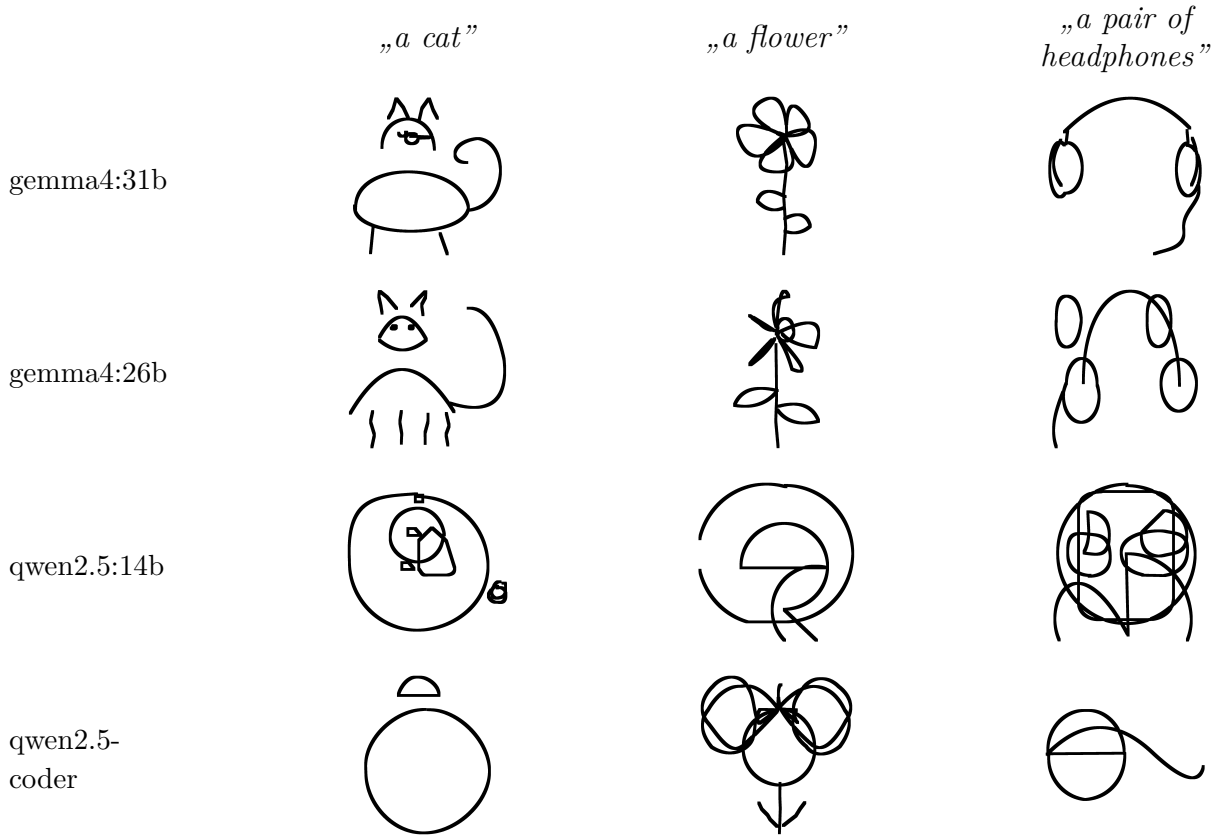


Figura 4.7: Comparație vizuală între cele patru modele pentru trei prompturi: „a cat”, „a flower” și „a pair of headphones”. Scorurile finale sunt sintetizate în Tabelul 4.2.

Model	Pisică	Floare	Căști
<i>gemma4:31b</i>	9	10	10
<i>gemma4:26b</i>	8	9	6
<i>qwen2.5:14b</i>	3	1	1
<i>qwen2.5-coder:14b</i>	3	7	3

Tabela 4.2: Scorurile finale ale modelelor în comparația pe trei prompturi.

Diferența dintre cele două familii de modelele e clară: *gemma4:31b* iese cel mai bine pe toate trei, *gemma4:26b* rămâne competitiv la pisică și floare dar slab la căști, iar modelele Qwen locale produc desene în mare parte de nerecunoscut (excepție floarea la *qwen2.5-coder:14b*, 7 / 10). De aici alegerea *gemma4:31b* pentru experimentele principale: bucla îmbunătățește doar un desen pornit dintr-o reprezentare interpretabilă, deci capacitatea de bază a modelului rămâne decisivă. Tocmai această fragilitate e cadrul potrivit pentru region lock: un model slab desenează nu doar prost, ci și *instabil*, riscând la fiecare reluare

să strice ce reuşise.

4.6 Efectul mecanismului de region lock

Region lock este mecanismul central al lucrării (Secţiunea 3.5): prin construcţie, orice cale pe care Criticul nu o vizează rămâne neschimbată între iteraţii. Cum păstrarea geometriei e garantată prin cod, întrebarea de evaluare nu este *dacă* păstrează, ci *cât de des* ar fi fost nevoie de el, adică de câte ori Artistul încearcă să suprascrie părţi deja acceptate.

La fiecare iteraţie, sistemul notează câte căi din afara regiunii-ţintă a încercat Artistul să modifice şi au fost readuse la forma anterioară (*preservation report*). Cifrele din Tabelul 4.3 nu sunt mici: la casă, Artistul a încercat de zece ori să redeseneze părţi deja acceptate, iar la bicicletă şi soare de câte şapte ori. Fără mecanism, fiecare încercare ar fi alterat geometria acceptată.

Prompt	Suprascrieri prinse de lock	Scor final
„a house with a roof, a door, two windows, and a chimney”	10	10
„a bicycle”	7	9
„a smiling sun”	7	10
„a crab with two claws, six legs, and two eyes”	4	9

Tabela 4.3: *Intervenţiile mecanismului de region lock în rulările principale (Artist gemma4:31b). Coloana centrală numără căile deja acceptate pe care Artistul a încercat să le suprascrie şi pe care lock-ul le-a readus la forma anterioară; rata de păstrare rămâne 100% prin construcţie.*

La modelele puternice, efectul lock-ului abia se vede în scor. Cu *gemma4:31b*, dezactivarea lock-ului (`redraw_all`) rar produce o degradare vizibilă: scorul rămâne monoton crescător în toate cele 11 prompturi cu lock şi coboară în doar 2 din 11 fără lock (casa $8 \rightarrow 5$, ceasul $7 \rightarrow 5$). Un model puternic redesenează competent şi de la zero, deci diferenţele rămân sub un punct, mai ales că evaluatorul nu penalizează complet incoerenţele geometrice (Secţiunea 4.7).

Lock-ul contează cu adevărat la un Artist slab. Ca să izolăm efectul, am reluat ablaţia cu un Artist mai slab (*gemma4:26b*) şi acelaşi Critic. Aici generarea e destul de instabilă încât consecinţele dezactivării lock-ului se văd cu ochiul liber. Figura 4.8 arată „a crab with two claws, six legs, and two eyes”: cu region lock, părţile acceptate rămân ancorate şi compoziţia se construieşte incremental; fără lock, fiecare iteraţie reface tot subiectul, iar corpul, cleştii şi picioarele migrează liber de la un pas la altul.

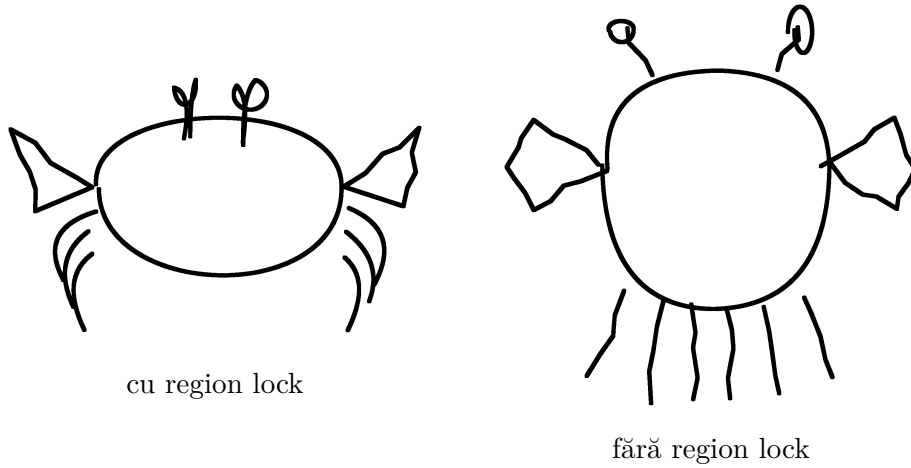


Figura 4.8: „a crab with two claws, six legs, and two eyes” desenat de un Artist mai slab (gemma4:26b). Cu region lock, părțile acceptate anterior sunt păstrate; fără lock, desenul este refăcut integral la fiecare iterație, deci structura poate migra între pași.

Cele două regimuri se completează: la un model puternic, region lock intervine rar și nu schimbă scorul (Tabelul 4.3), dar la unul slab diferența se vede limpede: cu lock, desenul rămâne coerent de la un pas la altul; fără, structura migrează (Figura 4.8). Valoarea mecanismului nu stă într-un câștig de scor garantat, ci în *garanția prin construcție* că progresul nu se poate pierde.

4.7 Sinteza rezultatelor și limitări

Tabelul 4.4 adună scorurile pentru cele opt prompturi principale. Șapte se îmbunătățesc pe parcursul buclei, iar racheta este acceptată din prima trecere.

Prompt	$t=1$	$t=2$	$t=3$	$t=4$	$t=5$	$t=6$	Δ
„a smiling sun”	6	5	8	8	9	10	+4
„a cat with a curled tail”	4	6	6	7	8	9	+5
„a coffee mug with a handle”	7	5	7	8	5	9	+2
„a pair of headphones”	5	5	6	7	8	10	+5
„a rocket with two fins, a round window, and flames”	10	–	–	–	–	–	0
„a house with a roof, a door, two windows, and a chimney”	5	5	5	8	9	10	+5
„a sailboat on waves”	7	7	7	8	9	10	+3
„a bicycle”	6	6	7	7	8	9	+3

Tabela 4.4: Scorurile Criticului pentru cele opt prompturi principale.

Traietoriile nu sunt mereu monotone (cana, de exemplu, scade temporar la $t = 2$ și $t = 5$), ceea ce reflectă cât de instabilă e percepția Criticului de la un pas la altul.

Evaluarea are și câteva limitări.

Întâi, Criticul este și evaluator, iar scorul se *plafonează* odată ce părțile cerute sunt prezente: nu penalizează degradarea structurii, deci două desene vizibil diferite pot lua

același scor (bicicleta, Secțiunea 4.2.4). De aceea evaluăm region lock-ul vizual și prin garanția prin construcție, nu prin diferențe de scor (Secțiunea 4.6): scorul spune dacă desenul e recognoscibil, dar nu și cât de bine e construit.

Apoi, numărul de prompturi ajunge pentru o analiză calitativă, dar nu pentru concluzii statistice. Nu în ultimul rând, rezultatele variază de la o rulare la alta (generarea și evaluarea sunt stocastice), iar comparațiile dintre regimuri sunt între traiectorii independente, nu între variante ale aceleiași rulări.

5. Concluzii și direcții viitoare

5.1 Concluzii

Lucrarea a propus și implementat o buclă Artist–Critic pentru generarea secvențială de desene vectoriale SVG. Contribuția centrală e mecanismul de *region lock* impus prin cod: garantează că orice parte nevizată de Critic rămâne identică între iterații, transformând o buclă instabilă într-una care progresează monoton. Evaluarea arată că bucla ajută față de o singură trecere, că region lock-ul intervine frecvent și că rezultatul depinde decisiv de capacitatea modelelor de bază: bucla valorifică modele care pot deja produce SVG coerent, dar nu acoperă golurile celor prea slabe.

5.2 Limitări

Pe lângă limitările evaluării discutate în Secțiunea 4.7, sistemul are două constrângeri structurale. Abordarea cere modele relativ mari pentru ambele roluri: pe hardware modest, modelele suficient de capabile scot o iterație în zeci de secunde, ceea ce face bucla lentă pentru uz interactiv. Obiectele cu topologie dificilă, precum bicicleta, rămân greu de desenat coerent chiar și cu modele puternice.

5.3 Direcții viitoare

Lucrarea poate fi dusă mai departe în mai multe direcții: specializarea unui model mic pe generarea de SVG (*fine-tuning*), ca să coboare pragul de hardware; *raționament geometric explicit*, ca desenele să respecte numărul exact de elemente cerut în prompt; un *evaluator independent* de Critic, care ar scoate conflictul de rol din evaluare; și extinderea interfeței într-o aplicație interactivă, în care utilizatorul să poată interveni el însuși în buclă.

Bibliografie

- [1] Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared Kaplan, Prafulla Dhariwal et al., „Language Models are Few-Shot Learners”, în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.
- [2] Alexandre Carlier, Martin Danelljan, Alexandre Alahi și Radu Timofte, „DeepSVG: A Hierarchical Generative Network for Vector Graphics Animation”, în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020.
- [3] David Ha și Douglas Eck, „A Neural Representation of Sketch Drawings”, în *arXiv preprint arXiv:1704.03477* (2017).
- [4] Ajay Jain, Amber Xie și Pieter Abbeel, „VectorFusion: Text-to-SVG by Abstracting Pixel-Based Diffusion Models”, în *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2023, pp. 1911–1920.
- [5] Haotian Liu, Chunyuan Li, Qingyang Wu și Yong Jae Lee, „Visual Instruction Tuning”, în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [6] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe et al., „Self-Refine: Iterative Refinement with Self-Feedback”, în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.
- [7] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger și Ilya Sutskever, „Learning Transferable Visual Models From Natural Language Supervision”, în *Proceedings of the 38th International Conference on Machine Learning (ICML)*, 2021, pp. 8748–8763.
- [8] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser și Björn Ommer, „High-Resolution Image Synthesis with Latent Diffusion Models”, în *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, pp. 10684–10695.
- [9] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser și Illia Polosukhin, „Attention Is All You Need”, în *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.
- [10] Ximing Xing, Juncheng Hu, Guotao Liang, Jing Zhang, Dong Xu și Qian Yu, „Empowering LLMs to Understand and Generate Complex Vector Graphics”, în *arXiv preprint arXiv:2412.11102* (2024).

- [11] Ximing Xing, Haitao Zhou, Chuang Wang, Jing Zhang, Dong Xu și Qian Yu, „SVGDreamer: Text Guided SVG Generation with Diffusion Model”, în *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.